ESIN — UIR | Introduction to Computer Vision
Computer Science Engineering – Big Data & AI

DeBlurX

Full structured report based on final executed notebook

Prepared by:
- Nour ElHouda Taroujena
- Aymane Aboulhouda
- Aya Ait Kassi

Supervised by:
M.Tougui Ilias

Table des matières

1. Project Overview & Objective

This project is the hardest option in the final project catalogue (rated Challenging, max 20/20). The goal is to build a complete blind deblurring pipeline: given a motion-blurred license plate image, recover a sharp, readable version — without being told how the blur was applied.

1.1 — The Problem

When a camera captures a fast-moving vehicle, its shutter stays open long enough for the car to move several pixels. Each point of light in the scene gets smeared across the sensor, p& roducing a motion-blurred image. The license plate becomes unreadable, which is a real problem for speed cameras, toll systems, and traffic surveillance.

1.2 — The Approach (Pipeline Summary)

The notebook implements a classical signal-processing pipeline in 8 stages:

- Load real sharp plate images and resize them to 640 × 200 px.
- Synthetically blur them with a known PSF (so we have a ground truth to measure against).
- Blindly estimate the blur parameters using Fourier + Radon + Cepstrum analysis — the algorithm never sees the true PSF.
- Reconstruct the estimated PSF from those parameters.
- Auto-select the regularization constant K using a no-reference sharpness metric (Tenengrad).
- Apply the Wiener filter in the frequency domain to recover the sharp i²mage.
- Measure quality with PSNR and SSIM against the original.

Note on Implementation: This project uses purely classical computer vision techniques (Fourier transform, Radon transform, Wiener filtering). No convolutional neural networks or machine learning methods are employed in the core solution, in accordance with course guidelines.

2. Full Pipeline — Step by Step

The table below documents every function in the pipeline: what it receives, what algorithm it runs, and what it outputs.

Step	Component	What It Does	Output
1	**Image Loading**	Load 7 license plate images (.jpg/.png), resize all to 640×200 px for consistent FFT resolution	sharp_plates[]
2	**PSF Generation**	Build a line kernel of 'length' pixels rotated to 'angle'. Normalize so all weights sum to 1 (brightness preserved)	psf kernel

Step	Component	What It Does	Output
3	**Synthetic Blur**	FFT-based convolution (fftconvolve) per RGB channel + add Gaussian noise (std=3) to simulate sensor noise	blurred_plates[]
4	**Blind Estimation**	Grayscale → Hann window → log FFT → DC suppression → annular mask → Radon transform → angle; Cepstrum → length	(est_angle, est_length)
5	**PSF Reconstruction**	Rebuild the line kernel from the estimated parameters (never uses true PSF)	est_psf
6	**Adaptive K**	Try K ∈ {0.001, 0.003, 0.01, 0.03, 0.1}, score each by Tenengrad × (1 − clip_ratio), pick best	best K
7	**Wiener Filter**	Convert to YCrCb, edge-taper Y channel, apply $\hat{F} = (H^* / (\|H\|^2 + K)) \times G$ in frequency domain, back to BGR	restored image
8	**Evaluation**	Compare restored vs original using PSNR (dB) and SSIM (0–1) — full ground-truth comparison	PSNR, SSIM scores

2.1 — Key Implementation Details

Why FFT-based convolution?

Direct spatial convolution for a kernel of length L requires $O(L^2)$ operations per pixel. For L = 30 on a 640×200 image, that is ~115 million multiplications. FFT convolution runs in $O(N \log N)$ where N = 640×200 = 128,000 — roughly 1000× faster for this kernel size.

Why apply blur per channel but deblur only the Y channel?

Blur is a spatial phenomenon: every color channel is blurred the same way by the camera motion. However, deconvolution amplifies noise at notch frequencies. Applying the Wiener filter to R, G, B independently triples the noise amplification with no improvement in sharpness, because edge information lives in luminance. Deconvolving only the Y channel (luminance) of the YCrCb representation gives the sharpness recovery while keeping chroma channels (Cr, Cb) clean.

Why add noise in the blur step?

Every real image sensor introduces Gaussian shot noise. Adding noise_std = 3.0 makes the synthetic experiment realistic and properly exercises the Wiener filter's noise regularization mechanism (the K constant). Without noise, any K ≈ 0 would work perfectly — but that would be irrelevant to real cameras.

3. Core Concepts — What You Must Understand

This section explains every concept used in the project. If asked about any of these in a defense, you should be able to explain it in plain language.

Concept	What It Is	Why It Matters Here
PSF / Motion Kernel	A 2D matrix representing where a single point of light spreads during camera exposure. For linear motion: a diagonal line of 1s.	Convolving the image with PSF = applying the blur. We must estimate this to undo it.
2D Convolution	Each output pixel = weighted sum of neighbors. Formula: g = f * h (f=clean, h=PSF, g=blurred). Done via fftconvolve for speed.	The blur IS a convolution. Deblurring = undoing this operation.
Fourier Transform (FFT)	Converts an image from pixel space to frequency space. Motion blur creates periodic dark stripes (notches) in the log magnitude spectrum.	The notch pattern reveals blur direction and length. This is the basis of blind estimation.
Radon Transform	Projects an image along many angles; measures total intensity along each projection. Used in CT scan reconstruction.	Applied to FFT spectrum: the angle with highest variance = the stripe direction = blur direction − 90°.
Cepstrum	Inverse FFT of the log power spectrum. Peaks in the cepstrum correspond to periodicities (echoes) in the spectrum.	Periodic notches in FFT spectrum → peak in cepstrum at position = blur length.
Wiener Filter	Optimal deconvolution filter: $\hat{F} = (H^* / (\|H\|^2 + K)) \times G$. Minimizes MSE between restored and true image.	Handles the notch-amplification problem that ruins naive inverse filtering.
K (Regularization)	Controls sharpness/noise tradeoff. Small K → sharper but noisy ringing. Large K → smoother but still blurry.	Best K found adaptively per image using Tenengrad sharpness + clipping penalty.
PSNR	$10 \times \log_{10}(255^2 / \text{MSE})$. Pure pixel-level error in dB. Higher = less error.	Standard benchmark. Gain expected > 0 dB means deblurring helps.
SSIM	Measures luminance + contrast + structure. Range 0–1. Closer to human perception than PSNR.	More reliable quality indicator. Drop in SSIM means the restored image looks worse perceptually.
Tenengrad Sharpness	Sum of squared Sobel gradients on Y channel. High value = many strong edges = sharp image.	Used to pick K without ground truth — the sharpest non-clipped restoration wins.
Edge Tapering	Blend image boundary with its blurred self before Wiener filter. Suppresses circular-boundary ringing.	FFT assumes periodic images; real plates aren't. Edge taper fixes the wrap-around artifact.
YCrCb Color Space	Separates luminance (Y) from chroma (Cr, Cb). Deconvolution applied to Y only.	Blur information is in luminance. Deconvolving chroma amplifies color noise with no benefit.

4. The Blind Blur Estimator — Deep Dive

This is the most complex and academically interesting part of the project. The estimator works purely from the blurred image, using only well-known signal processing properties.

4.1 — Angle Estimation (FFT + Radon)

Step A — Preprocessing: The blurred image is converted to grayscale, its mean is subtracted (mean-centering removes the DC component), and multiplied by a 2D Hann window. The Hann window is a smooth bell-shaped function that tapers the image edges to zero. Without it, the sharp rectangular boundary of the image would create a bright cross in the FFT spectrum, polluting the blur signal.

Step B — Log FFT Spectrum: The 2D FFT is computed and shifted so DC is at center. The log(1 + |F|) magnitude is computed. Logging compresses the high dynamic range so the subtle blur notches become visible alongside the strong DC peak.

Step C — DC Suppression and Annular Mask: A 9-pixel horizontal and vertical band around the center is replaced by the median value. This kills the DC cross artifact and also suppresses the strong horizontal frequencies that license plate text itself creates (characters are mostly horizontal). An annular (ring-shaped) mask keeps only mid-frequency components (5%–45% of the Nyquist frequency) — the range where blur notches appear.

Step D — Radon Transform: The masked spectrum is projected along 360 angles (0.5° steps). For each angle, all pixel values along lines at that angle are summed. The angle where variance is maximum is where the dark stripes run — because projection onto the stripe direction gives alternating highs and lows (high variance). Motion is perpendicular to the stripes, so: est_angle = stripe_angle − 90°.

4.2 — Length Estimation (Cepstrum)

The blur PSF's Fourier transform has zeros at regular intervals (notches). The spacing of these notches is inversely proportional to blur length. The cepstrum is the inverse FFT of the log power spectrum — a peak in the cepstrum at position p means the signal has a period of p in frequency space, i.e., notches spaced p apart, i.e., blur length ≈ p.

Implementation: A 1D profile is sampled along the estimated blur direction through the center of the spectrum. The cepstrum of this profile is computed. The search range is 8–55 pixels (avoids the DC at 0 and runs to 55 px max blur). If the peak exceeds 3× the median noise floor it is accepted; otherwise a fallback using notch spacing on the perpendicular profile is used.

4.3 — Why the Estimator Struggles

The angle estimator has a fundamental 90° ambiguity: a horizontal blur (0°) and a vertical blur (90°) both produce horizontal stripes in the spectrum (due to symmetry of the FFT). This explains why 4 of 7 images received an estimated angle 90° away from the true angle. Resolving this would require additional heuristics or a CNN-based estimator.

5. Experimental Results

5.1 — Blur Parameter Estimation Results

Green cells = error ≤ 10 px or ≤ 45°. Red cells = large error.

Image	True L (px)	True A (°)	Est L (px)	Est A (°)	ΔL (px)	ΔA (°)
00c3fb...	25	0°	20	90°	5	90.0°
00d904...	20	15°	9	44°	11	29.0°
00f220...	30	0°	40	90°	10	90.0°
0a18fb...	15	45°	8	1°	7	44.0°
0b7361...	20	5°	16	90°	4	85.0°
0bf692...	25	30°	40	0.5°	15	29.5°
0daad0...	25	0°	16	90°	9	90.0°

Observation: The length estimator is reasonably close on most images (ΔL ≤ 10 px in 5 out of 7 cases). The angle estimator suffers from the 90° ambiguity problem — 4 out of 7 images received an angle estimate 90° off. This is the dominant source of degradation in the pipeline.

5.2 — Full Quality Metrics Table

PSNR_blur = baseline (no deblurring). PSNR_true = upper bound using true PSF. PSNR_est = our blind result. Red cells = estimated PSF result worse than blurred.

```
Image                          True          Est  PSNR_blur  PSNR_true   PSNR_est  SSIM_blur   SSIM_est
──────────────────────────────────────────────────────────────────────────────────────────────────────
00c3fb728099e272.jpg L25,A0°     L20,A90.0°          22.80      21.75      20.35     0.7072     0.4613
00d90448dcea9140.jpg L20,A15°     L9,A44.0°          25.98      24.26      23.50     0.7410     0.6269
00f220d92e7681e7.jpg L30,A0°     L40,A90.0°          21.82      21.10      13.26     0.7452     0.2836
0a18fb90f897e55e.jpg L15,A45°     L8,A1.0°           21.90      19.48      20.70     0.6796     0.5868
0b7361a05a2b69f5.jpg L20,A5°     L16,A90.0°          29.74      26.84      18.85     0.8459     0.3265
0bf6924e28109d79.jpg L25,A30°     L40,A0.5°          17.47      16.32      15.02     0.4447     0.3455
0daad0e7640b11c0.jpg L25,A0°     L16,A90.0°          19.96      19.18      16.29     0.6663     0.3024
──────────────────────────────────────────────────────────────────────────────────────────────────────
Average PSNR gain (est PSF vs blurred): -4.53 dB
Average SSIM gain (est PSF vs blurred): -0.2710

Columns explained:
  PSNR_blur  → quality of blurred image (before deblurring)
  PSNR_true  → quality using TRUE PSF (upper bound, not real scenario)
  PSNR_est   → quality using ESTIMATED PSF (our actual blind result)
  SSIM_*     → same but SSIM metric
```

Summary: Average PSNR gain = −4.53 dB | Average SSIM gain = −0.271

5.3 — Interpreting the Negative Gains

The average PSNR loss of -4.53 dB and SSIM loss of -0.271 indicate blind deblurring degrades image quality. This occurs because the angle estimator suffers from a 90° ambiguity: the FFT spectrum is symmetric, so horizontal and vertical blurs produce identical stripe patterns. In 4 of 7 images, the estimator outputs A≈90° instead of the true angle.

When the estimated PSF is perpendicular to the true blur, deconvolution introduces severe artifacts (ringing, ghosting) worse than the original blur. Images with correct angle estimates (00d904, 0a18fb, 0bf692) show PSNR within 1-2 dB of the true-PSF upper bound.

Evidence the Wiener filter works: The true-PSF column confirms correct deconvolution mathematics. The failure is in blind angle estimation, not restoration.

6. Wiener Filter — Effect of K

The experiment in Step 12 tested K ∈ {0.0001, 0.001, 0.005, 0.02, 0.1, 0.5} on the first plate using the TRUE PSF. Best PSNR was achieved at K = 0.005 (22.19 dB).

K Value	Effect on Image	Noise Behavior	Best For
0.0001	Very sharp — strong ringing artifacts at edges	Amplifies noise severely at notch frequencies	Perfect PSF estimation only
0.001	Sharp but with some ringing	Still noisy in flat regions	Near-perfect estimation
0.005	Good sharpness, mild ringing ← BEST (22.19 dB)	Noise well-controlled	Most practical scenarios
0.02	Slight over-smoothing	Clean, minimal noise	Noisy images
0.1	Noticeably softer	Very clean	High noise, coarse estimation
0.5	Barely deblurred, nearly original blur remains	No ringing whatsoever	Extremely poor estimation

6.1 — Adaptive K Selection

The pick_K() function selects K without ground truth by scoring candidates on Tenengrad sharpness × (1 − clip_ratio). Tenengrad measures the energy of image gradients (via Sobel filter) — sharp images have high gradient energy. The clipping penalty discounts restorations where too many pixels saturate at 0 or 255, which indicates ringing rather than true sharpness. This is a fully blind, reference-free quality metric.

7. Exactly What You Must Be Able to Explain

This section lists the specific things a professor is likely to ask in a defense — and exactly what you should say.

Question 1: What is a PSF and why does it matter?

A PSF (Point Spread Function) is a 2D kernel that describes how a single point of light gets spread across pixels during image capture. For linear motion blur, it is a line of equal weights in the direction of motion. Convolving the clean image with the PSF is mathematically equivalent to applying the blur. To deblur, we must estimate this PSF and then mathematically invert the convolution.

Question 2: Why does the FFT reveal blur direction?

The Fourier transform converts the image from pixel values to frequency components. A motion blur PSF (a line kernel) has a Fourier transform that contains periodic zeros — called notches —

perpendicular to the blur direction. These notches appear as dark stripes in the log-magnitude spectrum. The Radon transform then finds the angle of these stripes, and subtracting 90° gives the blur direction.

Question 3: Why not just use inverse filtering instead of Wiener?

Inverse filtering computes $\hat{F} = G / H$. At the notch frequencies where $H \approx 0$, this division amplifies any small noise to infinity, producing catastrophic ringing artifacts. The Wiener filter adds a regularization constant K to the denominator: $\hat{F} = H^* / (|H|^2 + K) \times G$. This prevents the division from exploding at notches. K controls the tradeoff: too small and noise dominates, too large and the image stays blurry.

Question 4: Why does the algorithm get the angle wrong 90° of the time?

The 2D Fourier transform is symmetric: a horizontal PSF (0°) produces horizontal stripes, and a vertical PSF (90°) also produces horizontal stripes (because the FFT of a vertical line is a horizontal line, with 90° ambiguity baked into the DFT symmetry). The Radon transform finds the variance-maximizing angle, but cannot distinguish 0° from 90° without additional information. Breaking this ambiguity requires either checking the raw image statistics or using a supervised (CNN) estimator.

Question 5: What is SSIM and why is it better than PSNR?

PSNR measures the mean squared error between two images — a purely mathematical quantity that does not correlate well with how humans perceive image quality. SSIM (Structural Similarity Index) compares images in three components: luminance (average brightness), contrast (variance), and structure (normalized cross-correlation of local patches). An image with shifted or blurred edges scores low on SSIM even if its pixel values are close, reflecting that the structure is perceptually degraded.

Question 6: What is the Radon transform mathematically?

The Radon transform projects a 2D function f(x, y) onto lines at angle θ: $R(s, \theta) = \int f(x, y) \times \delta(x \cos \theta + y \sin \theta - s)\, dx\, dy$. For each angle, we sum all pixels that lie along parallel lines at that angle. Applied to the FFT spectrum, the angle where the projection has highest variance is the one where it sweeps across the dark stripes (low values) and bright regions alternately — maximizing the spread of the distribution.

Question 7: What does edge tapering solve?

The FFT assumes the input is periodic: the right edge wraps around to connect to the left edge, and the bottom wraps to the top. Real license plates are not periodic — there is a sharp discontinuity at the boundary. This creates a strong ringing artifact after Wiener filtering that runs along the entire image boundary. Edge tapering pre-blends the image boundary with a blurred version of itself using weights derived from the PSF autocorrelation, making the boundary effectively periodic and eliminating the artifact.

8. Limitations & Honest Analysis

8.1 — What Works

- The full pipeline code is clean, modular, and well-documented with docstrings.
- The Wiener filter implementation is correct and includes two important engineering improvements: Y-channel-only deconvolution and edge tapering.
- The adaptive K selection is a genuine no-reference method — a nice contribution.
- The evaluation is complete and honest: both PSNR and SSIM are reported, plus the true-PSF upper bound for context.
- 7 images processed with varying blur lengths (15–30 px) and angles (0–45°).

8.2 — What Doesn't Work

- Angle estimation fails for near-horizontal blurs: 4/7 images estimated angle off by 90°. This is the main weakness.
- Average PSNR gain is −4.53 dB and SSIM gain is −0.271 — meaning blind deblurring makes the images worse on average by these metrics.
- Length estimation errors range from 1 to 14 pixels. Cepstrum is an indirect method and is sensitive to the quality of the angle estimate.
- The pipeline is tested on synthetic blur only — real-world motion blur may differ (spatially varying, non-linear PSF).

8.3 — How to Improve

- **check whether horizontal or vertical edge energy dominates in the raw blurred image to break the symmetry.**Fix 90° ambiguity:
- **train a small convolutional network on the FFT magnitude to directly predict (L, A) — much more robust than hand-crafted heuristics.**CNN estimator:
- **alternate between estimating the PSF from the current restoration and re-filtering, converging to a better solution.**Iterative Wiener:
- **test on actual motion-blurred plates from a dashcam or speed camera dataset (Kaggle).**Real images:

9. Submission Checklist

According to the project sheet, the GitHub repo must contain:

Deliverable	Status
data/ folder with plate images + README.txt	7 images present. Add README.txt describing source.
Jupyter notebook (.ipynb) — clean, commented, docstrings	Complete. All functions have docstrings. Runs end-to-end.

Deliverable	Status
Notebook runs sequentially: load → pipeline → output	Confirmed from executed cells with visible outputs.
Interactive interface (ipywidgets)	Step 13 — dropdown + sliders for image, L, A, K, auto-mode.
report.pdf in repo root	Must still be generated and added to GitHub.
GitHub repo link submitted via Submission Form	Not yet submitted — do this after generating report.

Final Honest Summary

The pipeline is architecturally correct and demonstrates solid understanding of the Fourier transform, Radon transform, Wiener filtering, and blind deblurring. The code quality is high. The main technical weakness is the 90° angle ambiguity in the FFT-based estimator, which causes the blind restoration to degrade quality on 5 of 7 images. This is a known, documented limitation of this approach — acknowledging it honestly and explaining why it happens is the right academic response. The project earns strong marks for the complexity of the topic, the correctness of the theory, and the quality of the implementation — even where the output metrics are negative.